The Statistical Grimoire: Modern Statistical Approaches for the Natural Sciences Using R

Jeffrey M. Pisklak

2026-03-20

Table of contents

About **3**

Why This Book Exists **4**
Foolish Assumptions Made by Your Author 4
Errata 5

1 Introduction to R **6**
1.1 What is R? 6
1.2 The Genesis of R 6
1.3 Why a Progamming Language? 6
1.4 Why R? 9

2 How to Use R on a Computer **11**
2.1 Installing the R Programming Language 13
2.1.1 Windows Installation 14
2.1.2 macOS Installation 15
2.1.3 Linux Installation (Ubuntu and Mint) 16
2.2 Installing an IDE (RStudio and Positron) 19

References **20**

About

The Statistical Grimoire is a beginner-friendly open-access textbook actively being updated with statistical and data analysis techniques using the R programming language and other open-source tools.

This book was created to give students a solid, practical, and free introduction to both statistics and R programming, especially if they have not done any coding before.

> **Caution**
>
> You're reading a what amounts to a friendly beta version (it literally only has one complete chapter so far). Check back for additions and improvements.

The Statistical Grimoire: Modern Statistical Approaches for the Natural Sciences Using R by Jeffrey M. Pisklak is licensed under Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International.

https://github.com/statistical-grimoire/book

This is a Quarto book. To learn more about Quarto books visit https://quarto.org/docs/books.

This book uses the Braille Institute's Hyperlegible family of fonts. To learn more visit: https://www.brailleinstitute.org/freefont/

- **Body Typeface:** Atkinson Hyperlegible Next
- **Code Typeface:** `Atkinson Hyperlegible Mono Font`

Why This Book Exists

Many students arrive at graduate level study with a strong dislike of statistics and, importantly, without the preparation modern research demands. The gap here isn't only a statistical one. It's also about the tools and workflows that make statistical work reliable, transparent, and reproducible.

Open-source tools such as R, Git, Markdown, and LaTeX, and many others are now standard in many research settings. Students paying for a rigorous education should be introduced to these tools early. Yet many never even learn that they exist, and those that do typically discover them by accident since most programs barely mention them. Those who do encounter them often learn under pressure, quickly and inconsistently, because a deadline leaves them with no alternative. That path is stressful and prone to mistakes. Its the route I and many others took and it left me frustrated and resentful. How could an overpriced education fail so badly at preparing students for the practical realities of contemporary research.

These tools are not distractions from learning statistics; they support the process. They help students compute, document, verify, and communicate their work with far less friction than would otherwise be experianced. In an era of unprecedented access to computing power, it makes little sense to train students as if that power, and the practices that harness it, barely matter. When we cling to older methods by default, we risk mistaking tradition for rigor and blaming students for problems our training choices create.

This book is an attempt to close that gap: to teach statistics alongside the open-source tools and workflows that students will actually use, so they can spend less time fighting the process and more time doing careful, credible research.

The primary tool used throughout this book is the R programming language, though we will introduce other tools (markdown, git, LaTeX) when appropriate. Over the long term, my goal is for this text to function as a practical, open-access guide that leads complete beginners step by step through both statistical thinking and statistical programming in a clear, thorough, and usable way. Please treat everything here as a work in progress: the book will evolve.

Foolish Assumptions Made by Your Author

- You will actually read this (yes, I'm an optimist)

- You probably don't like math. You might even fear it, but deep down, you know it's good for you. Math, like confession, is rarely pleasant but often necessary.
- You'll start at the beginning and move forward.
- You won't take the sillier stuff too seriously—especially the parts that sound true. They probably aren't. Probably.

Errata

No manuscript is free from error. Should you unearth any mistakes, please file an issue on GitHub:

https://github.com/statistical-grimoire/book/issues

1 Introduction to R

1.1 What is R?

R is a programming language. A progamming language is simply a language humans use to communicate instructions to a digital computer.

1.2 The Genesis of R

The name "R" was derived from the first initials of its original two programmers, "R"obert Gentleman and "R"oss Ihaka. The decision to name the language using a single English letter is what might, charitably, be called a joke on the part of these two programmers, who saw themselves poking fun at R's parent language, which was given the unimaginative name of "S". In the 1970s the S language had undergone its initial development at the famous Bell Laboratories with the primary aim of enabling and encouraging "GOOD DATA ANALYSIS"- a goal so fundamental to the ethos of S that the authors, Becker and Chamber (1984), felt they had to emphasize it using uppercase lettering inside the preface to the language's inaugural instruction manual (the uppercase lettering has been reproduced here for the reader's benefit). The familial correspondence R has with S is present even to this day, to such an extent that Becker and Chamber's original manual could probably function decently well as an introductory manual to R itself.

1.3 Why a Progamming Language?

At this point readers might be wondering why it should ever be necessary to learn a programming language to conduct statistics and data analysis more generally. These topics are usually considered difficult enough by many students and educators, what need is there to compound this with a programming language? Why not, for instance, make use of any one of the many pieces of statistical software that already exist and (as the advertising would want you to beleive) require no requisite knowledge of programming? In other words, why not use software

such as SPSS[1] or one of its many malformed, and equally expensive, doppelgängers, Minitab, SAS, and Stata.

The primary answer to this question lies in flexibility. There is rarely a single correct way to analyze data, as different datasets come with their own unique challenges and intricacies. These complexities often resist the rigid, prescriptive approaches employed by many proprietary software programs. This is not to say software like SPSS cannot adapt to such scenarios—it often can. However, this adaptation typically comes at a cost: users may need to pay for additional features not included in the original purchase, or they may face an even steeper learning curve. One that forces mastery of a obscure and enigmatic language. A language that is so specific to the software, that only a select few (if any) even seem to truly understand it.

In direct contrast to this, R offers an intuitive and empowering experience for users. While it may seem daunting at first, R operates in a straightforward and logical manner, much like a calculator. Many users discover that using R is far easier than they initially expected. This is largely due to the vibrant and dedicated R community that exists online, which has cultivated an extensive network of resources over the years. Acolytes of R see it as something worthwhile to preserve and develop (often at their own personal time and expense).

Proprietary statistical software has no equivalent to this, nor will it ever. Users are often snared within its ecosystem not out of preference or love for the program, but because it is all they have ever known. Moreover, contrary to what their marketing might lead you to believe, the learning curve for these programs is dangerously steep, and users are unknowingly at risk of being lead off a cliff.

Owing to its nature as a programming language crafted for statistics, R is a language grounded by the logic of mathematics. This foundation often makes it easier for new users to understand and build upon, even for those who claim to dislike math. Moreover, proficiency with R grants users the ability to work with other statistical software if needed. The reverse, however, is rarely true: mastering SPSS or similar programs does not provide the same level of flexibility or transferable skills.

An altogether different answer to the question that opened this section, and one that will appeal to the University students reading this, is simply cost. R is free for the user, with no need to put up with annoying advertising or pay for additional features. The same can not be said of the other aforementioned software which are almost always subscription based, requiring the user to consistently renew a very expensive license to use the software. But R is not just free in monetary terms, it is also free in philosophical terms. R adopts the Free Software Foundation's GNU General Public License and thus adheres to the philosophy of *free software* (what some might term *open-source*).

[1] SPSS is popular software for conducting statistics that was originally released in the late 1960s and is an acronym for Statistical Package for the Social Sciences. At some point it was purchased by IBM and re-branded to mean *Steeply Priced Shitty Software.*

! The four essential freedoms

From the GNU project website (Free Software Foundation 2026):
A program is free software if the program's users have the four essential freedoms:

- The freedom to run the program as you wish, for any purpose (freedom 0).
- The freedom to study how the program works, and change it so it does your computing as you wish (freedom 1). Access to the source code is a precondition for this.
- The freedom to redistribute copies so you can help others (freedom 2).
- The freedom to distribute copies of your modified versions to others (freedom 3). By doing this you can give the whole community a chance to benefit from your changes. Access to the source code is a precondition for this.

This philosophy extends beyond the software itself to include both its file formats and help documentation. For years, the dissemination of scientific findings has been (and still is) hindered by the reliance on proprietary file formats imposed by commercial research tools. Locking information within these exclusive systems is clearly counterproductive to scientific progress, as it binds researchers to overpriced, branded ecosystems. Such practices prioritize profit over the broader goals of accessibility and collaboration, making their continued adoption ethically questionable. In practical terms, this means that choosing R is not just about its functionality, it is also a statement against the restrictive and exploitative behaviours proprietary software providers perpetuate. More plainly, and if for no other reason, we should use R just to give the middle finger to these companies.

As if you didn't need any other reasons to start using R immediately, here are some more:

- **R Is Not A Gooey Mess:** Unlike programs tied to a graphical user interface (GUI, often called "gooey"), R is not limited by point-and-click constraints. Its capabilities are as vast as what you and others can program (and your computer can handle).
- **Advanced Statistical Capabilities:** R's packages make it easy to apply best practices in statistics.
- **Enhanced Data Visualization:** With intuitive tools like *ggplot2*, *Shiny*, *Plotly*, and many others, R easily permits sophisticated and customized visualizations.
- **Reproducible Research:** R is built for reproducible research, aligning with open-science principles. It allows you to create scripts that are easy to share, review, and rerun, by anyone. This helps ensure transparency, accuracy, and reliability.
- **Integration with Other Tools:** R can easily integrate with other software and programming languages, such as Python, SQL, HTML, LaTeX, and even Excel. This makes it a valuable tool for working in diverse computational environments. Moreover, because of it's nature as programing language, version control is seemlesly applied with systems like Git.

While the prospect of learning a programming language like R may seem daunting at first,

it ultimately provides a more adaptable, intuitive, ethical, affordable, and rewarding tool for statistical analysis than many of its proprietary counterparts.

1.4 Why R?

At this point, it is worth addressing a question that comes up often: "Why use R instead of something like Python or Julia?" It is a fair question. After all, Python and Julia—like R—are full-fledged programming languages that are powerful and capable, but the difference comes down to origin story.

R is not a general-purpose language. It is the progeny of S, a language birthed in the depths of statistical practice for one primary goal: GOOD DATA ANALYSIS. Everything about R—from its object types to its default printing behaviour—is tailored to the sorts of things data analysts do every day.

By contrast, Python and Julia are general-purpose languages. They are designed to do many things well: build websites, run simulations, automate tasks, and yes, analyse data. But this means that good data analysis is a goal these languages aspire to, not one they were born to achieve.

To illustrate, suppose you wanted to calculate the mean of the numbers 1 to 100. In R, this is as natural as a reflex:

```
mean(1:100)
```

```
[1] 50.5
```

No need to import packages. No need to loop. No need to define arrays. It just works.

In Python, you will find that a bit more ceremony is required:

```
x = list(range(1, 101))
print(sum(x) / len(x))
```

```
50.5
```

Additional packages, like Python's excellent *numpy* package can simplify what needs to be written, but it still is not quite so good as what R offers as a baseline user experience.

Julia sits somewhere in the middle. It was built with scientific computing in mind, and its syntax can often be just as clean as R's:

```julia
using Statistics
mean(1:100)
```

However, Julia still expects you to opt in to statistics, with basic functions like `mean()` not being available until you explicitly load them. That is not a flaw BTW—it's a philosophical choice.

R, by contrast, assumes you are doing data analysis. You do not have to ask for permission to compute a mean. Moreover, neither Julia nor Python have the mature ecosystem of statistical tools, diagnostic plots, or nuanced modelling features that R does—at least not yet. R's statistical packages, often written by the very people developing the methods, remain second to none.

That being said Python and Julia are still great choices for data analysis and nobody should be discouraged from using them (to the contrary in fact).

2 How to Use R on a Computer

R will install and run straightforwardly on Windows and Macintosh operating systems as well as Linux;[1] however, prior to attempting any install it is important to make a simple distinction first. R is a programming language, which means it is nothing more than a language you use to communicate instructions to a computer. To communicate those instructions, some type of interface is required. This is a basic reality that applies to any language. It is quite difficult to communicate with someone if they have no mouth, eyes, or ears to send and receive communications with. Computers are no different in this respect. Simply understanding the language is not sufficient. For this reason, most operating systems come equipped with a basic way of interfacing with the user via a "terminal window" (see Figure 2.1). This is an application that allows you to type instructions (a.k.a. "commands") to your computer. Put more simply, when you see people "hacking" in movies, a terminal window is usually what they typing into. On computers using the Windows 11 operating system, this is referred to as the *Windows Terminal*, on Macintosh and Linux computers this is simple the *Terminal* application.[2]

For historical reasons, we refer to these applications "terminals" because they are mimicking specific technology of the past. In a bygone era, "terminals" were peices of computer hardware, consisting mainly of a keyboard paired with a monitor or printer, that people could use to send data to and receive data from a computer. Prior to this, information would have to be entered via punch-card (Norsk Teknisk Museum 2014).

Relying on your operating system's terminal application as a primary interface is often a cumbersome and inefficient experience, and definitely not a recommended course of action - though, for what it is worth, Linux users seem to delight in this sort of thing.[3] The preferred means of communicating R to your computer is via the use of what, in programming lingo, is commonly termed an *environment* or, more garrulously, an *integrated development environment* (IDE). This is simply a software application providing the user with a more polished visual workspace and feature set to make programming a smoother experience. IDEs exist for almost every language and for nearly any use case you can imagine. Among R users, RStudio and Positron are two of the most popular options. That said, IDEs vary widely in style and

[1] Admittedly, the process is probably a little less straightforward on Linux, but still managable.

[2] It's worth noting that most popular mobile and desktop operating systems do their absolute best to keep terminal applications out of sight. They they offer too much fine grain control of the computer (a.k.a. freedom). That makes it hard to herd users into the unquestioned patterns of behaviour the platform wants. You'll hear this justified as protecting the "user experience," but it rings hollow: 10-year-olds in the early '90s navigated MS-DOS to launch Duke Nukem, and '80s kids taught themselves BASIC to play Ultima. The assumption that users can't handle a command line isn't about safety, it's about control.

[3] I say that as someone who worships at the throne of Tux, the Penguin Emperor.

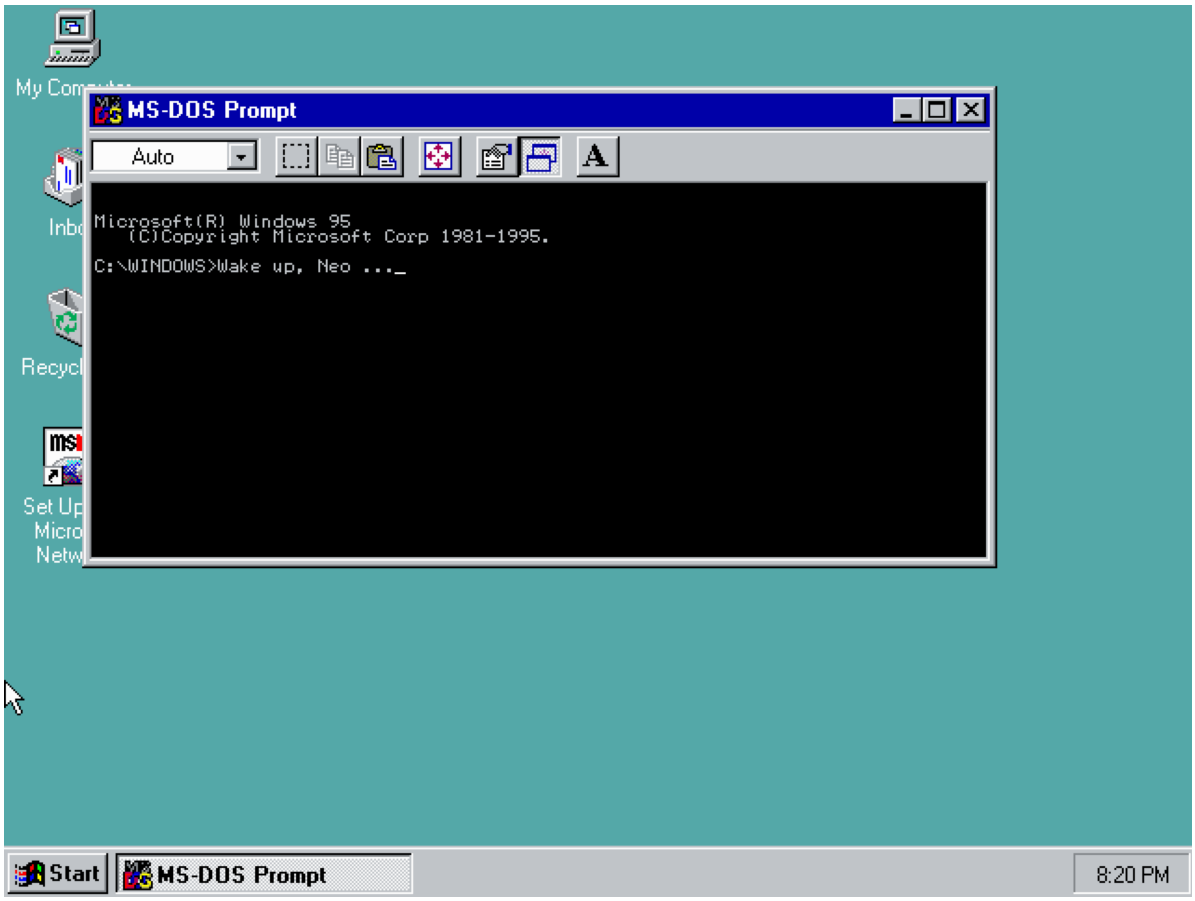

Figure 2.1: The MS-DOS Prompt application in Windows 95 is an example of a terminal window. This allowed users to access the DOS command line which was the foundation of the Windows operating system.

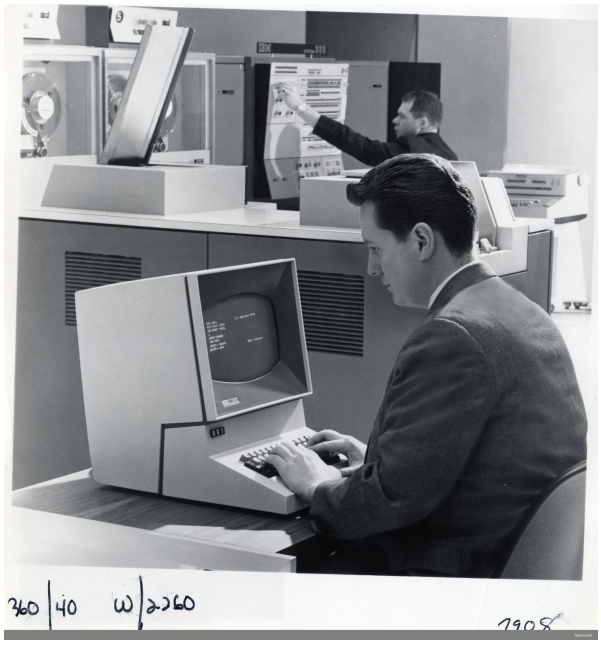

Figure 2.2: IBM 2260 terminal used to communicate with an IBM mainframe computer in the 1960s and 1970s.

capability, and using R does not require the use of these IDEs specifically, or any IDE for that matter. Still, installing one alongside the language itself is probably a good idea. However, before doing that, it is important to install the R language itself first.

2.1 Installing the R Programming Language

The standard installation of R will come with an associated environment for the user - provided they are working with either a Windows or Macintosh operating system. An example of the stock environment R ships with on Windows can be seen in Figure 2.3. Admittedly, as IDEs go, this will get the job done, but offers a pretty lackluster experiance. But don't worry, that's what RStudio and Positron are for.

To install R - both the language and environment simultaneously - simply go to the *R Project for Statistical Computing* website: https://www.r-project.org/ The site makes no apologies for its spartan '90s aesthetic, and we should all be so principled. Additionally, you'll encounter lots of technical jargon throughout the site's various pages, and as a newcomer to R and programming, some (maybe even a lot) of it will feel opaque. That's not a problem and you don't need to feel bad about it. Skim through it, absorb what clicks, maybe look up one or two things, and let the rest wash over you. Doing this builds familiarity with the landscape and overtime, as your skillset and vocabularly grows, the jargon will become more transparent.

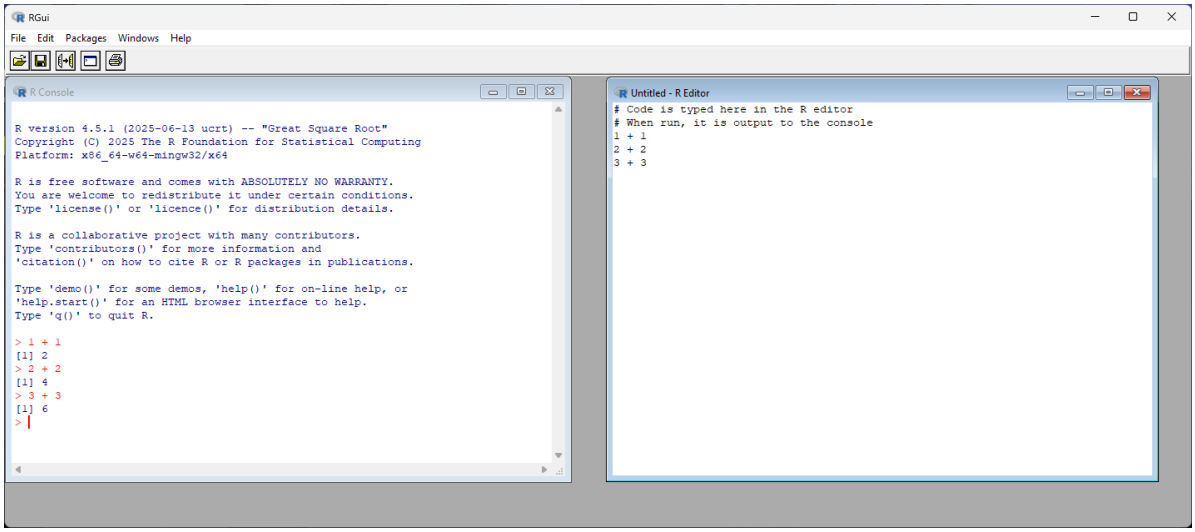

Figure 2.3: Base R environment for Windows, featuring the default console interface for running R code and viewing outputs (left) and the default scripting window (right) for writing, running, and saving R code.

Located on the front-page of this website should be a link labelled CRAN. This stands for "Comprehensive R Archive Network" and is a set of servers around the world that distribute R alongside software packages associated with R. The servers are "mirrored," meaning they all provide the same content. So there is no need to worry about one server providing incomplete, out-of-date, or unofficial versions of R. Technically speaking, the server closest to your home location is the one you should opt to download from; however, the topmost link labeled "0-Cloud" will be sufficient for most users. The install file is only about 80 megabytes large, so unless you live in the remotest areas of Earth, download speed, and thus choice of server, is probably not a concern.

Once you have chosen a suitable server, you will need select your operating system from the list the site provides (Chromebooks and Mobile devices are out of luck - though cloud-based alternatives do exist that can run R through a web-browser).

2.1.1 Windows Installation

Click the "Download for Windows" link to access the R download page (see Figure 2.4). Select the **"base"** option (labeled "install R for the first time"), which will direct you to the installer for the latest version of R. You'll see a download link for the current release—for example, **Download R-4.5.3 for Windows** - which is the newest version available at the time of writing.

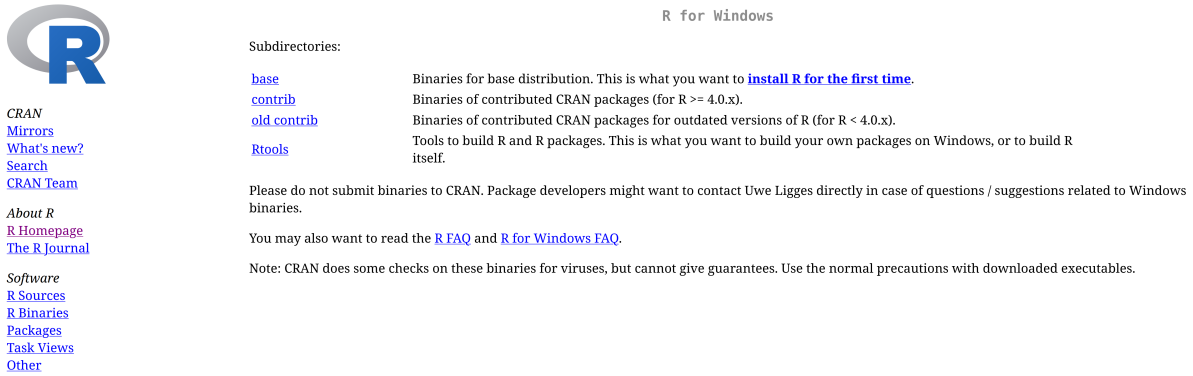

Figure 2.4: Windows download options

Once you have downloaded the (.exe) installation file (check your *Downloads* folder if you are unsure where on your computer it is located), open it and follow the setup wizard. Keep the default settings unless you have a specific reason to change them. Once installation is complete, you can launch R from the Start menu or a desktop shortcut (if you chose to create one during setup).

2.1.2 macOS Installation

Click the "Download R for macOS" link to access the R download page and select the option relevant to your computer (see Figure 2.5). At the time of writing this, Macintosh computers have recently begun being manufactured using their own in-house built processors (i.e., dubbed "Apple silicon"); however, many older Macintosh computers (pre-2023) still contain Intel-made processors. The install file you select will need to be determined by which type of processor your computer is using. Macintosh users can determine this by selecting *About This Mac* via the small little apple logo in the top left corner of their desktop screen. Machines using Apple silicon, will display a row called "Chip" and state something akin to "Apple M1". If you see this, you should opt to choose the install file that says "arm64". Machines using Intel processors will display a row reading "Processor" followed by the make and model of the processor. In this latter case, you should choose the install file that says "x86_64."

Once you have downloaded the (.pkg) installation file (check your *Downloads* folder if you are unsure where on your computer it is located), open it and follow the setup wizard. Keep the default settings unless you have a specific reason to change them. Once installation is complete, you can delete the install file and launch the base-R environment from the "launchpad" or applications folder.

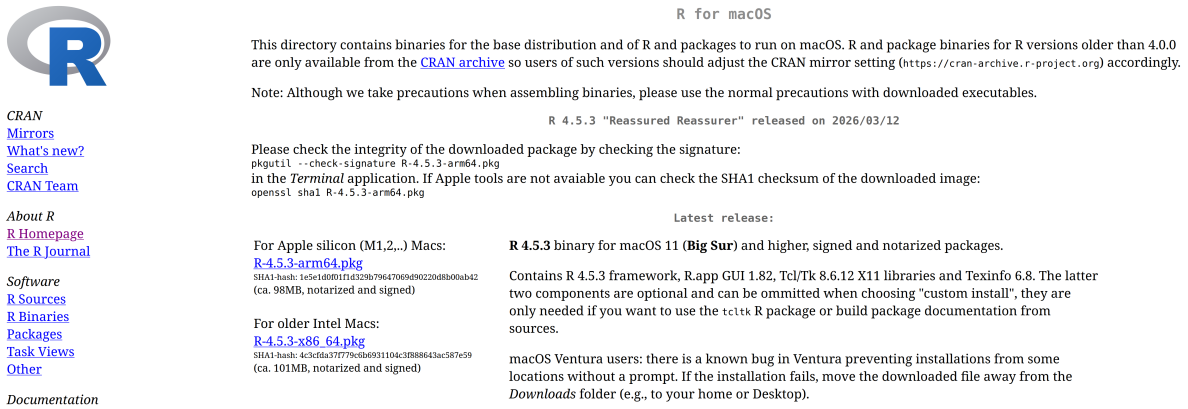

Figure 2.5: macOS download options

2.1.3 Linux Installation (Ubuntu and Mint)

Experianced users of Linux are not likely to need any instructions on how to install R on to their system; however, inexperianced users may need a helping hand as the install process is done entirely via the terminal. There are no install files you can download from a website and launch with a couple clicks of the mouse.

When it comes to Linux distributions (a.k.a. Linux operating systems), there are of course too many to even name. This makes it unrealisic to cover the install process for all of them. Consequently, this section will focus on two common beginner friendly distributions: Ubuntu and Linux Mint.

> **❗ Important**
>
> Unlike Windows and macOS, installing R on Linux does not include a graphical coding environment (see Figure 2.3). Consequently, Linux users must use the terminal to run R until they install an IDE such as RStudio or Positron.

2.1.3.1 Ubuntu

Many Linux distros, like Ubuntu, have their own officially currated repositories that you can download R from. However, the version of R these store often lags quite a bit behind the offical files supplied by the R Project's CRAN repository. The reason being, Ubuntu's package maintainers need time to test software, ensure it integrates well with other software on Ubuntu, and verify it works across different Ubuntu versions. This deliberate process keeps things stable but, unfortunately, means you're not getting the latest version of R immediately.

By contrast, if you need the latest R version or specific recent features/bug fixes, you can add the R Project's own repository to Ubuntu's package manager configuration (`apt`).

2.1.3.1.1 Installing from Ubuntu's Repository

If using Ubuntu's offical repository appeals to you, inside your terminal you can just run …

```
sudo apt update
sudo apt install r-base
```

2.1.3.1.2 Installing from the R Project's Repository (CRAN)

If, on the other hand, you would like the most recent releases of R and R packages (this is the reccommended approach), you will first need to download the GPG public key file from the R project's official server. This is used to verify that R packages from the the CRAN's Ubuntu repository are legitimate and haven't been tampered with.

In your terminal run the the following to first refresh your package lists:

```
sudo apt update
```

This tells `apt` "go fetch the latest list of available packages and their versions" from all the configured repositories on your computer.

Next we are going to install two helper programs, `software-properties-common` and `dirmngr`. The former provides tools for managing software repositories. The latter is background service that manages keys and certificates. It's used to verify GPG signatures on packages and keys.

```
sudo apt install --no-install-recommends software-properties-common dirmngr
```

Then, in order to install the GPG key file, we run …

```
 wget -qO- https://cloud.r-project.org/bin/linux/ubuntu/marutter_pubkey.asc | sudo tee -a /et
```

Next we add the R Project's repository to your system's package sources so `apt` knows where to find R packages. To do this, you need to determine the codename of Ubuntu you are running. Ubuntu names each new release of its operating system with an animal codename (focal = 20.04, jammy = 22.04, noble = 24.04, etc.) To determine the code name just run `lsb_release -cs` in the terminal.

In the code below you need to replace `noble` with the codename of your version of Ubuntu.

```
echo "deb https://cloud.r-project.org/bin/linux/ubuntu noble-cran40/" \
 | sudo tee /etc/apt/sources.list.d/cran-r.list
```

And now we can finally install R:

```
sudo apt update
sudo apt install r-base
```

And, for good measure, install build tools for installing R packages from source This is needed for many CRAN package installs.

```
sudo apt install r-base-dev
```

Check the version of R you have installed.

```
R --version
```

To run R simply type:

```
R
```

To quit running R:

```
q()
```

2.1.3.2 Linux Mint

Linux Mint is based on Ubuntu, so the installation process is identical, with one exception. Mint has it's own distinct codenames. This means in the steps above where we replace `noble`, we will need to make sure that we use the Ubuntu codename that your version of Mint is based on. This can be obtained by running the following in the terminal.

```
. /etc/os-release
echo "$UBUNTU_CODENAME"
```

Everything else about the Mint install, is the same as Ubuntu.

2.1.3.3 Other Linux Distributions

The offical R Project website contains detailed information for installing on other major Linux distributions. https://cloud.r-project.org/bin/linux/

2.2 Installing an IDE (RStudio and Positron)

As mentioned earlier, the default R environment that ships with a Windows and macOS install of R is quite limited (see Figure 2.3). While perfectly functional, it lacks many quality-of-life features found in modern IDEs. RStudio and Positron, by contrast, are the two most feature-rich IDEs for R, with RStudio having dominated the community for years and Positron emerging as a compelling newer alternative.

As of writing this, RStudio is the established choice. It has extensive documentation, countless tutorials, and broad support across the R community. By contrast, Positron is a much more recent development offering a more flexible and responsive interface built on Visual Studio Code's foundation.[4] Both applications are developed by Posit, a company focused on open-source tools for data science and scientific research. For a detailed feature comparison, see Posit's comparison page.

If you can't decide which to choose and want definitive choice made for you, go with Positron. It integrates modern development practices more seamlessly than RStudio, which was designed in 2011 when the computing landscape looked rather different. Both are excellent, but Positron won't make you feel like you're using legacy software.

Coming Soon:

- Installation Instructions for R, RStudio, Positron.
- Cloud-based IDEs (no installation required)

[4] Visual Studio Code is a free open-source IDE that has become the standard choice for many developers across multiple programming languages.

References

Becker, R. A., and J. M. Chambers. 1984. *S: An Interactive Environment for Data Analysis and Graphics*. Belmont, CA: Wadsworth.

Free Software Foundation. 2026. "What Is Free Software?" https://www.gnu.org/philosophy/free-sw.html.

Norsk Teknisk Museum. 2014. "22.0 IBM - Modell 360 / 370." Photograph. Norsk Telemuseum. September 10, 2014. https://digitaltmuseum.no/011015240147/22-0-ibm-modell-360-370.